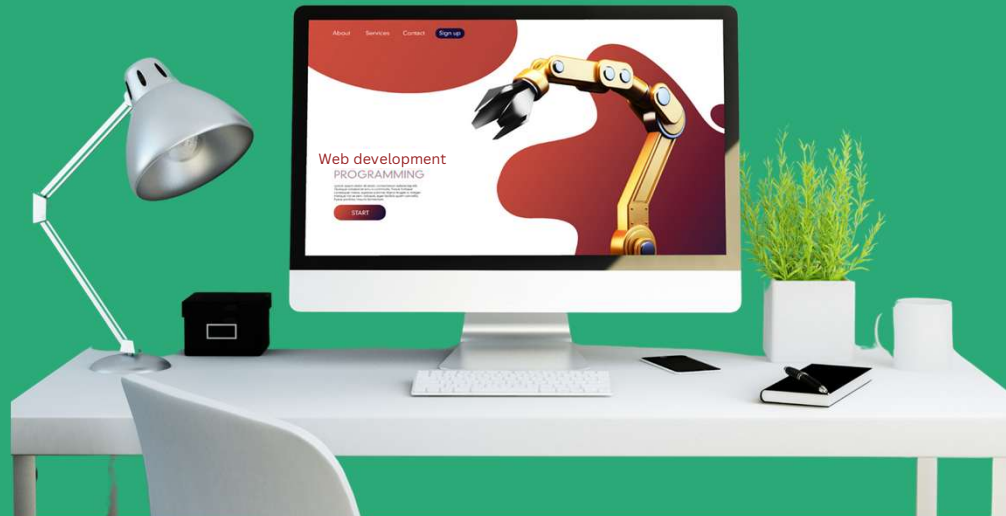


Full Stack Web Development with Python



NADA MAREY

Teaching Assistant &
Career Development Specialist



Information Technology Institute - ITI

i This presentation is protected by copyright.

Takeaway Notes

- We don't assume you have any previous knowledge
- You don't have to memorize the code; use your logic
- It is important to learn how to search for information

Where to search:

- Search Google
- Search StackOverflow
- ChatGPT & DeepSeek & ...



- If you have any questions during the lecture, don't hesitate to stop me and ask.

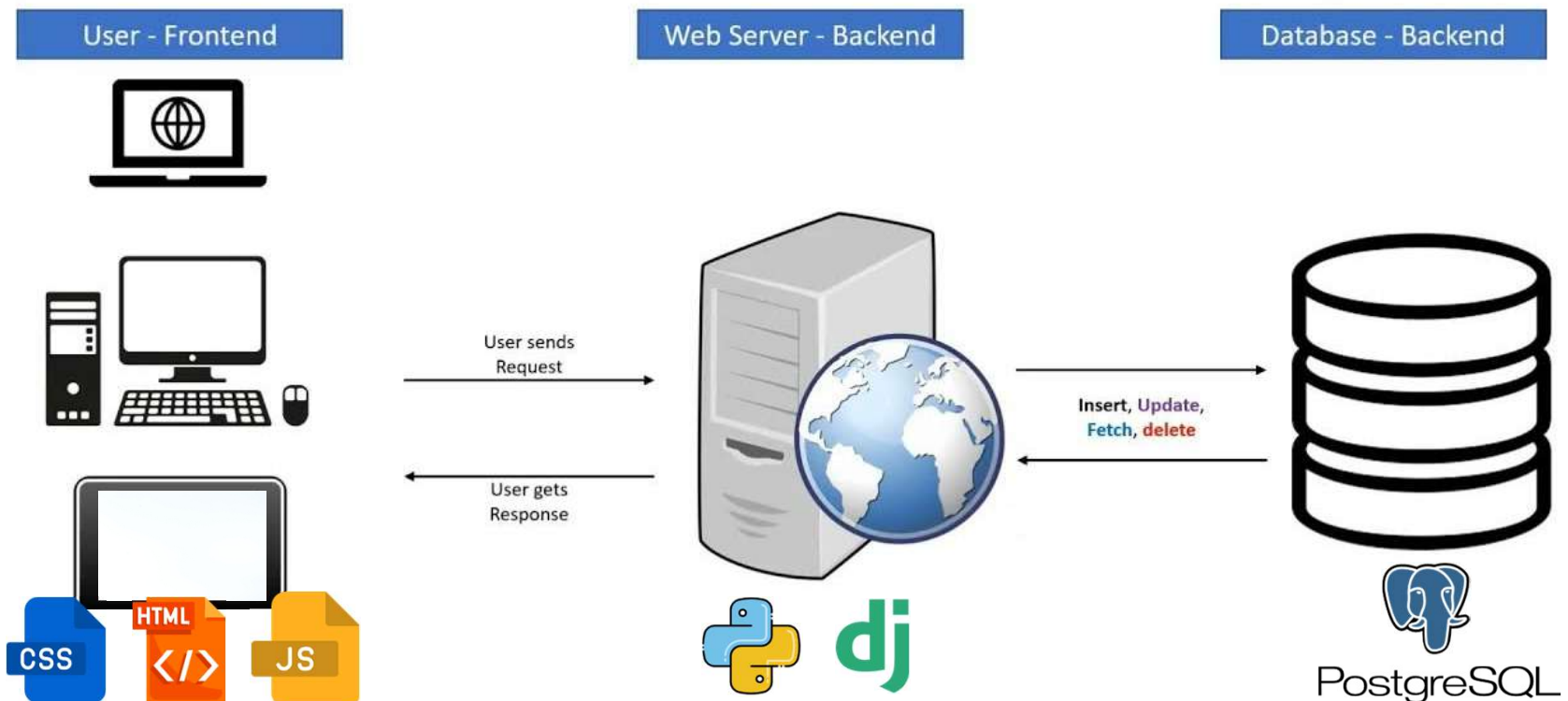
Full Stack Web Development with Python

We will learn:

- Python Object Oriented Programming (OOP)
- Python Frameworks (Django)
- Client-side Technologies (HTML & CSS & JS)
- Introduction to PostgreSQL database



Full Stack Web Development with Python



Django^{dj}

Python web framework

Intro

django



**Open Source
Framework**



**Building Web
Applications**



**Easy, Secured
and Fast.**

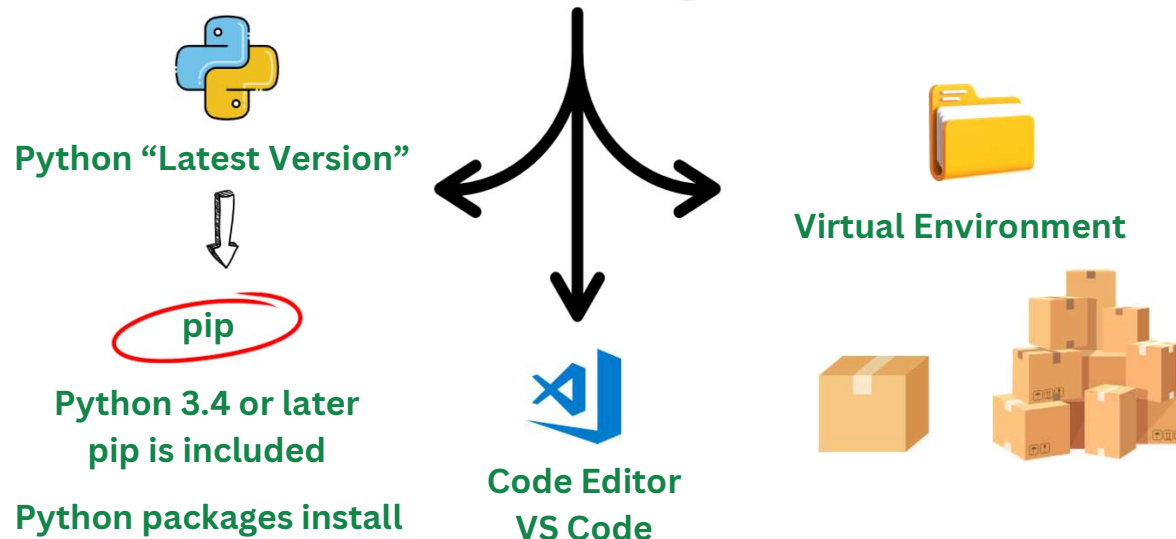


**Built with
Django**



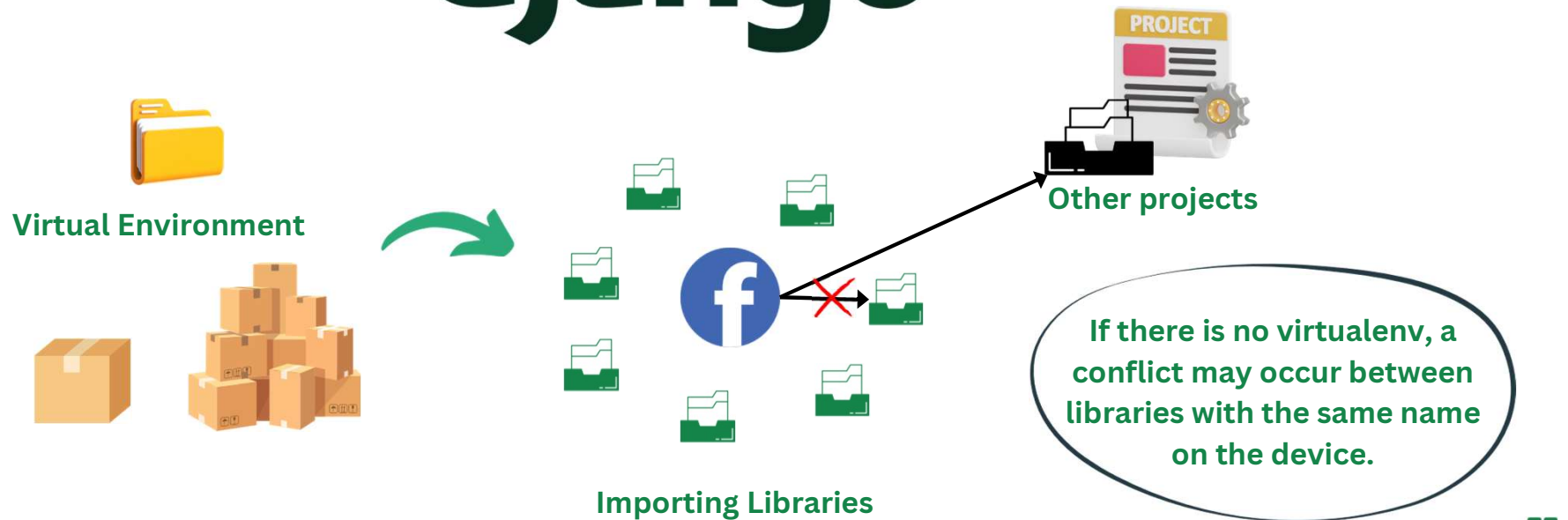
Intro

django



Intro

django



Part 1: Installation

Part 2: Requests and responses

Part 3: Models and the admin site

Part 4: Views and templates

Part 5: Forms and generic views

Part 6: Testing

Part 7: Static files

Part 8: Customizing the admin site

Intro

django

Some Knowledge

Much Practice

Build GitHub Repo



Django^{dj}

Installation

Setup virtualenv & Django Project

Step 1: Installation.

First, ensure you have Python installed. Download it from python.org if you don't have it already.

Step 2: Create a New Directory for your Django Project.

```
mkdir my_django_project  
cd my_django_project
```

Step 3: Create a Virtual Environment.

First install virtualenv if you don't have it:

Open the terminal → pip install virtualenv

Then create the environment:

In the terminal → virtualenv test OR python -m venv test

Step 4: Activate the Virtual Environment.

```
venv\Scripts\activate
```



Setup virtualenv & Django Project

Step 5: Install Django in the Virtual Environment.

`pip install django`

Step 6: Create a New Django Project.

`django-admin startproject myproject .`

Step 7: Run the Development Server

`python manage.py runserver`

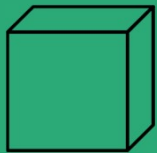
Step 8 : Deactivate the Virtual Environment

`deactivate`



virtualenv

Create Virtualenv



Activate Virtualenv



Install Django

django

Create Project



Run Server



Django^{dj}

Project files

Project files

`django-admin startproject 'NAME'`

`__init__.py` file: An empty file that tells Python that this directory should be considered a Python package.

`settings.py` file: Configuration for this Django project

`urls.py` file: The URL declarations for this Django project; a “table of contents” of your Django powered site.

`asgi.py` file: An entry-point for ASGI-compatible web servers to serve your project.

`wsgi.py` file: An entry-point for WSGI-compatible web servers to serve your project.



Django^{dj}

How does it work?

PROJECT

django-admin startproject 'NAME'

__init__.py file

settings.py file

urls.py file

asgi.py file

wsgi.py file

FACEBOOK

PROJECT = FACEBOOK

The PROJECT NAME IS FACEBOOK

WEB APPLICATIONS 

APP
GROUPS

TEMPLATES
VIEWS
URLS

APP
PAGES

TEMPLATES
VIEWS
URLS

APP
MARKET

TEMPLATES
VIEWS
URLS

APP
PROFILE

TEMPLATES
VIEWS
URLS

APP 

VIEWS

LEADER



TEMPLATES

FRONT-END



MODELS

DATABASE



URLS

PATH



Django^{dj}

Root directory files

Root directory files

1. `manage.py`:

- The command-line utility for administrative tasks in Django
- Used to run the development server, create migrations, run tests, etc.
- Commands like `python manage.py runserver` or `python manage.py migrate` start here

2. `db.sqlite3`:

- The SQLite database file that Django creates by default
- Stores all your application data when using SQLite (Django's default database)
- In production, you'd typically replace this with PostgreSQL, MySQL, etc.



Django^{dj}

Startapp

Startapp

Django's "MTV" (Model-Template-View) pattern, where:

Models handle data and business logic (database).

Templates handle presentation (Front-end).

Views handle request/response coordination.



Startapp

init.py:

- Makes the directory a Python package

admin.py:

- Where you register your models to include them in Django's admin interface
- Allows you to customize the admin interface

apps.py:

- Contains configuration for this specific app
- You can customize how Django treats this application

models.py:

- Defines your database models (the structure of your database tables)
- Each class typically represents a database table

tests.py:

- Where you write unit tests for your application
- Can be split into a tests/ directory as your test suite grows

views.py:

- Contains the view functions/classes that handle web requests and return responses
- The "controller" in MVC terms, handling business logic



Django^{dj}

Requests & Responses

App_Views

```
views.py

from django.shortcuts import render
from django.http import HttpResponse
# Create your views here.

def index(request):
    return HttpResponse("Hello, world. You're at the pages index.")
```

```
urls.py

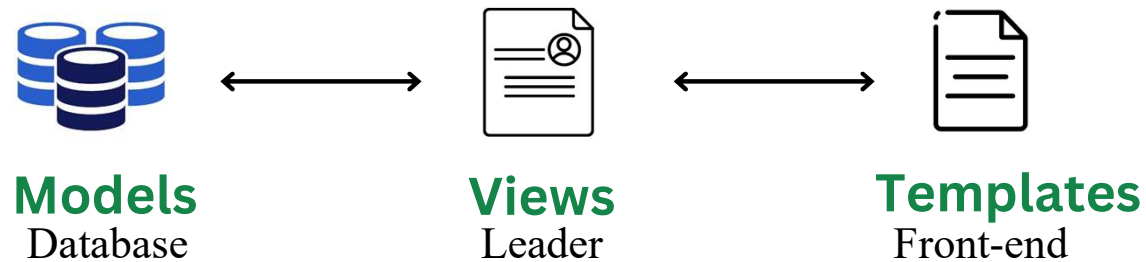
from django.urls import path
from . import views

urlpatterns = [
    # three inputs
    # 1. The URL pattern ('' means the root URL)
    # 2. The view function to call (views.index), index is a function defined in views.py
    # This function will handle requests to the root URL.
    # 3. A name for the URL pattern ('index')
    # This allows us to refer to this URL in templates and other parts of the code.
    path('', views.index, name='index'),
]
```



App_Templates

Step 1: How does it work?

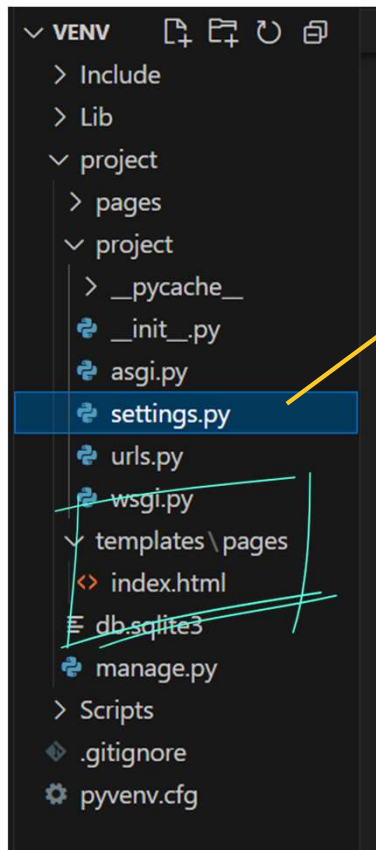


Step 2: Learn Django Templates Language (**DTL**)

Step 3: Creating the Template



App_Templates



```
settings.py

import os
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```



App_Templates

If you want to display a new page in the template to the user, follow these steps:

Step 1: Create an HTML page in the *Template folder*.

Step 2: Create a function for that page in the *Views*.

Step 3: Create a link to that page in the *URLs*.



App_Templates

```
views.py

from django.shortcuts import render
from django.http import HttpResponse
# Create your views here.

def index(request):
    return render (request, 'pages/index.html', x)
```

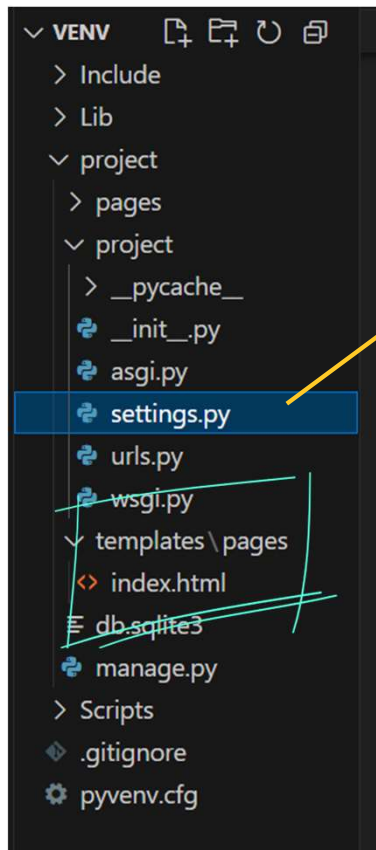
```
urls.py

from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
]
```



App_Templates



```
settings.py

import os
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```



Django^{dj}

Django Templates Language

“written inside the templates files”

Variables

```
views.py

from django.shortcuts import render
from django.http import HttpResponse
# Create your views here.

def index(request):
    # x is a dictionary that will be passed to the template
    # It contains the name and age of a pet
    # The template will use these values to display them on the page
    # The template is located at 'pages/index.html'
    # The render function takes the request, the template name, and the context dictionary
    x = {'name': 'Django', 'age': 3}
    return render (request, 'pages/index.html', x)
```

```
index.html

<h1> Hello, World</h1>

name = {{ name }}
<h1> age = {{ age }}</h1>
```



Filters

Filters are small functions that are applied on variables. They can be stacked by using a pipe character (|), *without space*. And they have arguments, denoted by a ":".

Some filters don't require arguments, some have optional ones, and others require them.

Examples

- Runs the "upper" filter on the content of the variable "var"

```
{{ var|upper }}
```

This construct runs the "upper" filter on the content of the variable "var" and then adds the string "foo" to the result of that.

```
{{ var|upper|add:"foo" }}
```



Filters

Here are examples of commonly used built-in Django template filters:

- **lower and upper:** These filters convert a string to lowercase or uppercase, respectively.
- **capfirst and title:** capfirst capitalizes the first character of a string, while title converts the string to title case (first letter of each word capitalized).
- **default:** This filter provides a default value if the variable evaluates to False (e.g., empty string, None, empty list).
- **length:** Returns the length of a string or the number of items in a list.
- **truncatewords and truncatechars:** These filters shorten a string to a specified number of words or characters, respectively, adding an ellipsis if truncated.
- **date and time:** These filters format datetime objects according to a specified format string.
- **add:** Adds the argument to the value. This can be used for numbers or to concatenate strings/lists.
- **join:** Joins elements of a list with a specified string.



Tags

Django template tags are special constructs used within Django templates to perform logic, control flow, and output dynamic content. They are enclosed within `{% %}`.

Here are examples of commonly used Django template tags:

- `extends`: Enables template inheritance, allowing a template to inherit from a base template.

Code



```
{% extends "base.html" %}
```



Tags

Here are examples of commonly used Django template tags:

- `block` : Defines content blocks that can be overridden in child templates when using `extends` .

Code



```
{% block content %}  
    <p>Default content for this block.</p>  
{% endblock %}
```

- `include` : Loads and renders another template within the current template.

Code



```
{% include "snippets/header.html" %}
```



Tags

Here are examples of commonly used Django template tags:

- `if`, `elif`, `else`: Controls conditional logic.

Code



```
{% if user.is_authenticated %}
    <p>Welcome, {{ user.username }}!</p>
{% elif user.is_staff %}
    <p>Hello, admin!</p>
{% else %}
    <p>Please log in.</p>
{% endif %}
```



Tags

Here are examples of commonly used Django template tags:

- `for` loop: Iterates over items in a list or iterable.

Code



```
{% for item in my_list %}  
    <p>{{ item }}</p>  
{% endfor %}
```



Thank you

